

Efficient Agentic LLM Serving over SSD-based Sparse KV Storage

Wenhao He^{*‡} Ping Zhang[‡] Xiaohe Hu^{§‡} Chutian Wang
Jinlong Hou[§] Yuan Cheng[§] Peng Sun[†] Fangcheng Fu^{*✉}

^{*}Shanghai Jiao Tong University [‡]Infrawaves

[§]Shanghai Innovation Institute [†]Shanghai Qiji Zhifeng Co., Ltd.

Abstract

Agentic sessions driven by Large language models (LLMs) often alternate between model inference and tool use, accumulating long histories across successive rounds. Serving these sessions efficiently requires reducing attention computation and retaining history key-value (KV) caches to avoid recomputation. Recently, frontier open-source LLMs adopt sparse attention to reduce computation by selecting only part of the history, while SSDs provide a cheaper alternative to CPU DRAM for storing KV caches. However, sparse KV selection depends on the ad hoc intermediate values during model inference, so it forces SSD reads to lie on the inference critical path. These reads are further slowed by fragmented accesses and read-write interference in SSDs.

To address these challenges, we present Janus, an agentic serving framework for sparse attention LLMs with SSD-centric KV storage. Janus focuses on append prefill, which processes each round’s newly added inputs and accounts for most history KV loading. To move SSD reads out of the critical path, Janus runs the model’s own KV selection module on earlier intermediate values, predicting KV demand without additional training. The predicted reads overlap with model computation, and any prediction misses are fetched before attention executes to preserve model outputs. To improve SSD efficiency, Janus coalesces adjacent reads, packs scattered KV pages into sequential writes on the CPU, and limits background writes while reads are active. Across three models and three agentic traces, Janus outperforms existing works by up to $1.57\text{--}3.69\times$ ($1.22\text{--}1.85\times$ on average) in terms of the time to first token latency, while maintaining decode efficiency.

1 Introduction

Large language models (LLMs) have advanced rapidly in reasoning, tool use, and code generation, making autonomous agents one of their most prominent applications. Unlike one-shot generation, an agentic session repeatedly inspects a

repository/directory, invokes tools, incorporates their outputs, and revises the solution. Consequently, agentic sessions contain long autonomous loops and increasingly long input contexts [53]. A recent study reports that agentic tasks consume three orders of magnitude more tokens than chatbots, with input tokens dominating the cost [2]. How to efficiently serve these long-running, multi-round sessions has therefore become an important systems problem.

The first essential property of agentic sessions is their long context. With full attention, processing a sequence of M tokens requires $O(M^2)$ attention computation, which grows prohibitively expensive as an agent accumulates the history context. Sparse attention reduces this cost by allowing each query to attend to only a subset of history tokens, while retaining global or local paths needed for model quality [48]. For example, top- k sparse attention uses a lightweight indexer to select $k \ll M$ history tokens and reduces the core attention cost to $O(Mk)$. This approach has moved beyond academic research and has now been used by frontier open-source LLMs, including DeepSeek-V4 and GLM-5 series models [6, 49].

The second outstanding property is repeated interaction. Each agent round appends a new user message, tool result, or model output to a much longer session history, and then performs an *append prefill* over the new tokens. Recomputing the history at every round would waste substantial GPU cycles. Thus, serving systems often retain the history key-value (KV) cache and reload it when the next append prefill query arrives [22, 27]. Due to the limited space of GPU high-bandwidth memory (HBM), current systems commonly offload KV caches to CPU DRAM. Recent systems further extend this design into a hierarchy spanning GPU HBM, CPU DRAM, and SSDs [37, 46].

Currently, predominant practices of KV storage are CPU DRAM-centric, treating SSDs or remote storage as lower hierarchies. However, such a design has become costly. Specifically, the ever-increasing context length and user sessions have demanded more and more storage capacity, which drives DRAM to be expensive as demand far exceeds supply. Worse still, the shortage of DRAM is expected to continue given

✉Contact: Fangcheng Fu <ccchengff@sjtu.edu.cn>

the rapidly growing state of agentic serving. In a snapshot of public server-component prices, a 256-GB DDR5 RAM costs \$8,500–\$14,350, whereas a 30.72-TB enterprise NVMe SSD costs \$14,500–\$16,000 [32–36]. The resulting per-GB cost differs by roughly two orders of magnitude. This gap has motivated SSD-centric KV storage like Tutti [28], which leverages GPU-SSD links for KV cache transmission, attempting to approach DRAM-centric serving performance while offering much larger capacity. Consequently, an SSD-centric design that minimizes reliance on CPU DRAM is appealing for cost-efficient agentic serving.

However, it is non-trivial to combine sparse attention with SSD-centric KV storage to serve agentic workloads efficiently. In particular, this work focuses on two essential challenges.

First, *sparse attention makes KV demand non-deterministic to the storage layer and shortens the opportunity to overlap KV loading*. Specifically, since SSD bandwidth is far below HBM bandwidth, SSD-centric systems prefer initiating KV loads early and overlapping them with model computation [22, 28, 46]. Dense attention makes this prefetch straightforward as all history KV caches will be consumed. In contrast, sparse attention determines the useful tokens only after its indexer processes the current query for KV selection. Waiting for the exact KV selection places SSD reads on the inference critical path. As we will show in §5, such SSD reads may occupy over 30% of the time cost of append prefill. Sparse attention therefore changes not only how many KV caches are read, but also when the serving system can know what to read.

Existing systems do not resolve this mismatch. Mainstream serving frameworks (e.g., vLLM and SGLang) load the full KV history to HBM so every token remains selectable [22, 45]. This approach maximizes compute–I/O overlap but causes redundant loading, wasting bandwidth and HBM capacity. Predictive approaches instead speculate which tokens will be selected and prefetch only those KV caches [15, 18, 20, 21, 51]. Nevertheless, most such designs target the LLM autoregressive decoding phase by exploiting selection similarity between adjacent decode steps, yet our empirical study in §3 reveals agentic workloads exhibit a different bottleneck: nearly all KV reads occur during append prefill, while decode mostly reuses history tokens loaded by append prefill or accesses newly generated tokens in the current round. The few designs that cover prefill [15, 20] either require offline model calibration or expose only a short overlap window that is insufficient for SSD latency (detailed in §3).

The second challenge is *the lack of SSD I/O optimization tailored for serving sparse attention LLMs under the agentic scenario*. Specifically, we consider two types of I/O issues, which are the non-contiguous I/O and the read-write interference. For one thing, it is common knowledge that SSDs are inefficient for small random accesses, yet serving sparse attention LLMs naturally produces such unfavorable accesses in both directions. On the read path, only the sparsely selected KV will be loaded, resulting in scattered SSD reads.

On the write path, LLM serving maintains newly generated KV caches in non-contiguous GPU pages due to the widely adopted paged attention technique [14], making offloading similarly fragmented. For another, SSDs suffer from the interference of reads and writes as they compete for device-internal resources [28]. Such interference has a larger impact on SSD reads compared to writes, deteriorating read bandwidth by nearly 60% when concurrent SSD writes exist. Nevertheless, agentic workloads exhibit significant read amplification due to the multi-round nature, which drives read volume to be tens of times greater than write volume (detailed in §3). Thus, the read-write interference is a particularly pressing problem for agentic workloads. Unfortunately, these I/O issues are under-explored in current systems.

In this work, we present Janus¹, an agentic serving framework for sparse attention LLMs under the SSD-centric KV storage paradigm. Janus addresses the uncertainty and limited overlap opportunity introduced by sparse KV selection, and the I/O issues related to SSD performance, keeping most KV loading off the critical path to make SSD-centric KV storage practical for long-running agent sessions.

First, Janus predicts sparse-attention KV selections early to create an overlap window for SSD loading. During append prefill, it reuses the LLM’s own indexer as a training-free predictor by feeding the indexer intermediate states available before its normal execution point. Janus then prefetches the predicted KV caches (i.e., predicted reads) while the main model continues computing. Once the model produces the exact KV selection, Janus fetches only the KV caches missed by the prediction (i.e., corrective reads), ensuring lossless model outputs. Moreover, as the prediction itself introduces additional computation, Janus further manages to accelerate it. Particularly, Janus partitions the history context across the participating GPUs and executes the prediction individually. This design has negligible impact on prediction accuracy, but prevents prediction from delaying the inference pipeline.

Second, Janus features a series of SSD I/O optimizations tailored for our targeted scenario. To address the non-contiguous I/O, Janus coalesces adjacent selected blocks through *best-effort read packing* to trigger contiguous GPU-SSD reads whenever possible, and uses *CPU-assisted write packing* to transform scattered GPU pages into sequential CPU-SSD writes with the help of CPU, avoiding competing with model execution. To address the read-write interference, we apply a *phase-aware write capping* to shelter read performance for agentic workloads.

We implement Janus and evaluate it using multiple agentic workload traces and long-context sparse-attention models. The results show that Janus limits SSD I/O exposed on the critical path to less than 6.5% of the append prefill latency, thereby reducing time-to-first-token (TTFT) and outperforming state-of-the-art baselines by up to $1.57\text{--}3.69\times$ ($1.22\text{--}1.85\times$

¹In Roman myth, Janus is usually depicted as having two faces, one looking backward to the past and the other looking forward to the future.

Table 1: Frequently used notations. Note that we list notations at token granularity for clarity. The same notation applies when one entry represents a token region.

Symbol	Description
$\llbracket n \rrbracket$	The set of indices $\{1, 2, \dots, n\}$.
P	Number of tokens in the cached history.
L	Number of tokens in the append prefill query.
$\mathbf{h}_t^{(\ell)}$	Input hidden states of token t at layer ℓ .
$\mathbf{z}_t^{(\ell)}$	Indexer key of token t at layer ℓ .
$\mathbf{k}_t^{(\ell)}, \mathbf{v}_t^{(\ell)}$	Key and value caches of token t at layer ℓ .
$\text{Idx}^{(\ell)}(\cdot, \cdot; k)$	Indexer’s function at layer ℓ .
$\mathcal{S}_t^{(\ell)}$	History tokens selected for token t at layer ℓ .

on average).

2 Preliminaries

This section briefly walks through the preliminaries related to our work. Frequently used notations are listed in Table 1.

2.1 LLM Inference and Serving

LLM inference consists of two phases. During the *prefill* phase, the model processes all input tokens in one step, computes their KV caches, and generates one output token. The model then enters the auto-regressive *decode* phase, which is executed for several steps, with each step processing the last generated token and generating one output token. In a multi-round session, the KV caches of the existing history can be reused, so only the newly appended tokens are processed. This operation is referred to as *append prefill*. Without loss of generality, we treat the initial prefill of each agentic session as an append prefill with zero history length.

The two phases have different workload characteristics. Prefill is typically compute-bound, especially for long context requests. Consequently, a single long append prefill can already saturate the GPU. In contrast, decode is typically memory-bound, as it repeatedly reads model parameters and KV caches while performing relatively little computation. Thus, modern LLM systems commonly apply the continuous batching technique [47] to batch multiple requests during decode, while executing each prefill individually.

2.2 Sparse Attention

Modern LLMs are mainly based on the Transformer architecture. In a nutshell, each Transformer layer comprises an attention block and a position-wise feed-forward network (FFN) block [40], as shown in Figure 1 (left). For clarity, we denote $\mathbf{h}_t^{(\ell)}, \mathbf{h}_t^{\prime(\ell)}$ as the hidden states for token t at layer ℓ ,

where the former is fed into the attention block and the latter is fed into the FFN block.

As the context grows, the attention mechanism becomes the primary bottleneck due to its quadratic complexity. Recent frontier LLMs use sparse attention to reduce the cost of processing long contexts [5, 6, 9, 49]. Compared to dense attention, sparse attention leverages a lightweight *indexer* to score the relevance between tokens, and then selects a fraction of tokens for the attention mechanism.²

Prefill phase. During an append prefill, each sparse attention layer first computes the selection for every appended token. The serving system combines these selections and loads the selected history KV caches into GPU HBM before executing core attention. To be formal, let P be the cached history length and L the appended query length. For each appended token at position $t \in \llbracket L \rrbracket$, the indexer computes its relevance scores to all preceding tokens and selects those with top- k scores, i.e.,

$$\mathcal{S}_t^{(\ell)} = \text{Idx}^{(\ell)}\left(\mathbf{h}_t^{(\ell)}, \left\{\mathbf{z}_s^{(\ell)}\right\}_{s \in \llbracket P \rrbracket}; k\right), \quad (1)$$

where $\mathcal{S}_t^{(\ell)}$ is the set of selected token indices for token t , $\text{Idx}^{(\ell)}(\cdot, \cdot; k)$ represents the indexer’s function, $\mathbf{z}_s^{(\ell)}$ is the indexer key for history token s . Indexer keys are per-token states rather than model parameters, and are stored along with KV caches. Before computing Eq. (1), the indexer keys for all history tokens must be loaded from the KV storage. Besides, the indexer keys for appended tokens are also derived from the hidden state $\mathbf{h}_t^{(\ell)}$ using the indexer’s parameters so that the token selection considers not only the history context but also the preceding tokens within the appended query. We omit such details here and refer interested readers to related technical reports [5, 6].

Subsequently, the per-token selections are combined into the union of selection, i.e., $\mathcal{S}^{(\ell)} = \bigcup_{t \in \llbracket L \rrbracket} \mathcal{S}_t^{(\ell)}$, which indicates the indices of all selected tokens for the current layer’s sparse attention. Before the attention computation, the KV caches of selected tokens $\{(\mathbf{k}_s^{(\ell)}, \mathbf{v}_s^{(\ell)})\}_{s \in \mathcal{S}^{(\ell)}}$ must be loaded to GPU HBM. The attention block produces intermediate hidden states $\mathbf{h}_t^{\prime(\ell)}$, which will be fed into the FFN block to compute the output hidden states of layer ℓ , namely $\mathbf{h}_t^{(\ell+1)}$.

In addition, the attention block also generates KV caches $(\mathbf{k}_t^{(\ell)}, \mathbf{v}_t^{(\ell)})$ for each appended tokens $t \in \llbracket L \rrbracket$. After append prefill finishes, the loaded KV caches for selected history tokens and the newly generated KV caches for appended tokens remain in GPU HBM for the ensuing decode phase.

Decode phase. During the auto-regressive decoding phase, each decode step performs sparse KV selection for the token processed at that step. The routine is similar to an append

²In DeepSeek-V4, sparse selection and attention are performed at the entry granularity, where each entry represents a token region (e.g., 128 consecutive tokens). By default, we describe the mechanism at the token granularity, while our work is implemented at the entry granularity for DeepSeek-V4.

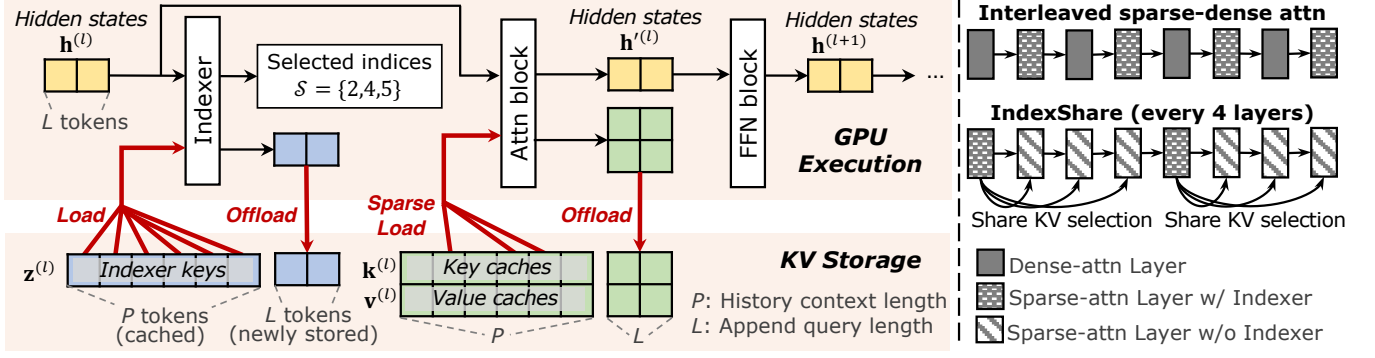


Figure 1: Left: Illustration of the sparse-attention Transformer architecture at layer ℓ . We omit the details (e.g., normalization and residual connection) in attention block (Attn) and feed-forward network block (FFN). Right: Illustration of the interleaved sparse-dense attention design and the IndexShare design that are used in frontier open-source LLMs.

prefill with a single appended token. If the selection contains a history token whose KV cache is not resident in GPU HBM, the serving system loads that KV cache before executing attention. Otherwise, there will be no KV cache loading. Each newly generated KV cache also remains in GPU HBM for subsequent decode steps.

Uses in frontier open-source LLMs. Frontier LLMs organize sparse-attention layers in different ways, as illustrated in Figure 1 (right). DeepSeek-V4 series models interleave sparse-attention layers with dense-attention layers [6]. Each sparse-attention layer performs its own indexer for KV selection, while two consecutive selections are separated by a dense-attention layer. GLM-5 series models instead adopt the IndexShare design: every four consecutive sparse-attention layers form a group, in which only the first layer executes an indexer and the remaining three layers reuse its selected token indices for their respective KV caches [49]. As we will show in §4.2, these two designs result in different KV selection performance, which our work takes into account.

2.3 KV Storage

LLM serving systems retain the KV caches of processed tokens so that subsequent requests can reuse the cached history instead of recomputing it. Because GPU HBM cannot hold the KV caches of many long-running sessions, serving systems commonly offload inactive KV caches to CPU DRAM or SSD and reload them on demand [22, 27, 28, 46]. Sparse attention LLMs additionally retain the indexer keys described above. Both indexer keys and KV caches are persistent session states. However, the loading of indexer keys is deterministic, so it can be prefetched to overlap the loading. In contrast, only the KV caches of selected history tokens are required, making it infeasible to exactly prefetch those KV caches before the indexer module finishes.

GPU-initiated SSD storage. To enhance I/O performance, it is desirable to transfer data directly between GPU HBM

and NVMe SSDs, bypassing the CPU. Notably, GPU Direct Storage (GDS) enables peer-to-peer direct memory access (DMA) between HBM and SSD, thereby avoiding a staging copy through CPU DRAM [26]. However, GDS still relies on the CPU to initiate each I/O request. In response to this, GPU-initiated SSD storage systems such as GeminiFS and Tutti expose NVMe queues to the GPU, allowing GPU kernels to submit I/O commands and observe their completion without per-request CPU intervention [28, 29]. The CPU remains responsible for setup and coarse-grained metadata management outside the data path.

In particular, NVMe communicates with GPU kernels through queue pairs, each pair consisting of a Submission Queue (SQ) and a Completion Queue (CQ). To issue an I/O, a GPU kernel enqueues a command in the SQ that describes the I/O request (e.g., the operation, source and destination addresses, and transfer length). The GPU then rings the SQ’s doorbell to notify the NVMe controller. The controller fetches the pending command(s), accesses the SSD, and transfers the data directly between SSD and HBM through peer-to-peer DMA. After the transfer finishes, the controller posts a completion signal to the CQ, which the GPU polls to determine when the requested I/O is done. This queue-based solution allows GPU kernels to submit asynchronous I/O requests and overlap storage access with GPU computation.

3 Observations and Motivation

Agentic serving differs from conventional multi-turn chat in both its input and output lengths. We first characterize this difference and then present two observations about its impact on SSD-backed sparse-attention serving, which motivate the design of Janus.

Length characteristics of agentic workloads. Conventional multi-turn conversations usually append a short user message and generate a longer model response. Agentic sessions often exhibit the opposite pattern. An appended query can contain

Table 2: Token-length distributions in three agentic traces (detailed in §5.1), where P is the history context length before an agent round, L is the append query length, and O is the output length in that round.

	Trace 1			Trace 2			Trace 3		
	P	L	O	P	L	O	P	L	O
Avg.	77K	1.1K	237	119K	1.4K	365	125K	4.8K	357
P25	71K	742	142	85K	727	222	67K	856	98
P50	77K	901	228	131K	1.3K	404	70K	2.2K	216
P90	85K	1.8K	401	154K	2.7K	540	323K	12K	671
P99	88K	2.3K	510	155K	4.0K	590	375K	22K	1.1K

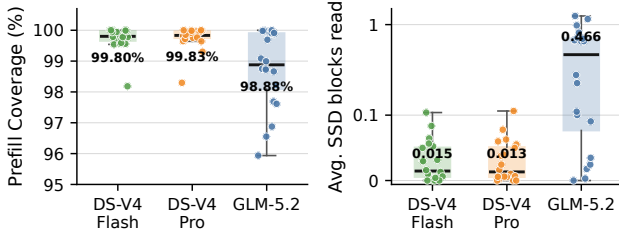


Figure 2: Comparison of KV loading between append prefill and decode over 20 agentic rounds, evaluated on DeepSeek-V4-Flash, DeepSeek-V4-Pro, and GLM-5.2. Left: The coverage of history KV caches that are required by decode phase but loaded in append prefill. Right: Average numbers of SSD blocks loaded per sparse attention during decoding.

the output of a tool, such as a PDF extractor or web searcher, or the result returned by a sub-agent. Such outputs can be much longer than a user message. Meanwhile, an agent decomposes a complex task into multiple steps, and each step often generates only a tool invocation, an intermediate decision, or a short partial result. Agentic workloads therefore commonly combine long inputs, short outputs, and many rounds.

Table 2 confirms this pattern in three agentic traces. Their median history lengths reach 77K, 131K, and 70K tokens, respectively. The append query length and model output length are relatively short compared to the history. However, the former is still 3-20 $\times$ of the latter in both median and P99 values. These distributions align with recent characterizations of coding-agent workloads, which report long contexts and short outputs [53].

This length asymmetry has two implications for KV storage. First, a thousand-token append query can require a large set of history KV caches during append prefill, whereas the short decode can reuse much of this set. Second, repeated agent rounds can load the history KV caches multiple times over the lifetime of a session. We next quantify these two effects.

Observation 1: Append prefill dominates KV loading. Both append prefill and decoding perform sparse KV selection and may load history KV caches, as described in §2.3. To identify the dominant phase, we record the history KV volume

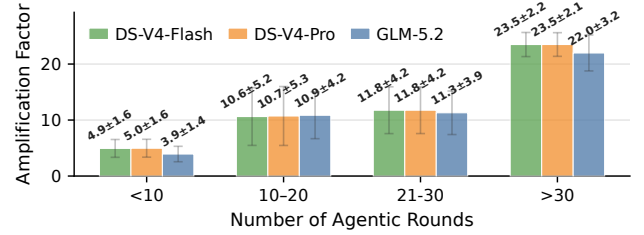


Figure 3: The read amplification factors (i.e., cumulative read volume divided by cumulative write volume) under different agentic rounds.

transferred from SSD to GPU HBM during representative agent rounds. We separately aggregate the transfers triggered by append prefill and all decode steps in the same round. Our measurement counts only actual history KV transfers and excludes KV caches already resident in GPU HBM and KV caches newly generated in the current round.

We randomly sample 20 agentic rounds from the traces and record the sparse KV loading during append prefill and the subsequent decoding steps. As shown in Figure 2, across the three models, roughly 99% of the history KV caches required by the decode phase are already loaded in append prefill. Even though decode often spans hundreds of steps, most decode steps trigger no KV loading at all. Instead, they reuse the history KV caches loaded during append prefill and the KV caches generated earlier in the same round.

The long-input, short-output pattern explains this concentration. An append prefill computes sparse selections for many appended tokens, and their union brings a broad history working set into GPU HBM. This working set remains resident during the ensuing decode phase. Subsequent decode steps therefore tend to select KV caches already in HBM or KV caches newly generated in the current round, leaving little additional data to load from SSD.

This observation shifts the optimization target from decoding to append prefill. Most existing KV prefetchers focus on decoding and predict future selections from their similarity across adjacent decode steps [18, 21, 45, 51], which cannot be leveraged in append prefill. There are few predictive prefetchers for append prefill, yet they still fail to provide both lossless execution and a sufficiently long SSD overlap window. InfiniGen [15] performs model-specific offline calibration so that attention scores can be estimated for KV selection. However, it requires specific model tuning and cannot guarantee the output to be lossless. ECHO [20] partitions prefill queries into segments, performs per-segment KV selection prediction, and overlaps the current segment’s prefetching (from CPU DRAM) with the next segment’s prediction. However, this requires the indexer computation could overlap sparse KV reads, which is infeasible for SSD storage since sparse KV reads take longer time than indexer (detailed in §5.2).

Observation 2: Agentic sessions amplify KV reads. The multi-round nature of agentic sessions creates *KV read amplification*. A token’s KV cache is written to SSD once after it is generated and evicted from GPU HBM. However, that KV cache can be selected and reloaded by multiple later append prefills as the agent accumulates tool results and intermediate decisions. Long appended queries further increase the history KV demand in each round. Consequently, the cumulative amount of KV data read from SSD can exceed the amount written to SSD over a complete session.

Sparse attention reduces the history KV cache loaded in each round, but does not eliminate this amplification. It limits each appended token to a subset of the history, yet the same history KV cache can still appear in the selected subsets of many later rounds. Figure 3 compares the cumulative KV reads and writes of sparse attention from real agentic traces. It can be seen that the total read volume remains 3.9-23.5 times of the write volume despite sparse selection. And the amplification enlarges as the number of agentic rounds increases.

KV read amplification makes SSD read efficiency particularly important compared to SSD write efficiency. Moreover, full-duplex data paths do not isolate reads and writes inside an SSD: concurrent requests still compete for device-internal resources and degrade I/O efficiency [28]. Sparse attention further scatters reads across the selected history, while paged KV allocation scatters writes across non-contiguous GPU pages [14]. These properties make the repeated reads induced by agentic sessions difficult to execute efficiently.

Together, our observations raise two important questions. First, how to hide the sparse KV loading during append prefill rather than decode, as it dominates the I/O exposed on the critical path. Second, how to emphasize read efficiency when optimizing SSD I/O for our target scenario owing to the read amplification of agentic workloads. These also motivate the core designs of Janus introduced in §4.

4 Janus

4.1 Overview

Figure 4 depicts the overview of Janus, which mainly consists of the execution plane and the storage plane.

The core of the execution plane is our KV selection prediction (§4.2). When an append prefill request arrives, Janus triggers the prefetching for deterministic KV loading (e.g., indexer keys and KV caches for dense attention layers), and adopts *training-free lookahead prediction* to facilitate prefetching of KV caches of sparse attention. Specifically, the KV selection of a target layer is predicted a few layers ahead, guided by a *model-aligned prediction distance*. In addition, to reduce prediction overhead, we develop *synchronization-free parallel prediction*, which parallelizes prediction while removing cross-GPU synchronization. Based on the prediction results, GPUs trigger predictive reads to prefetch the sparsely

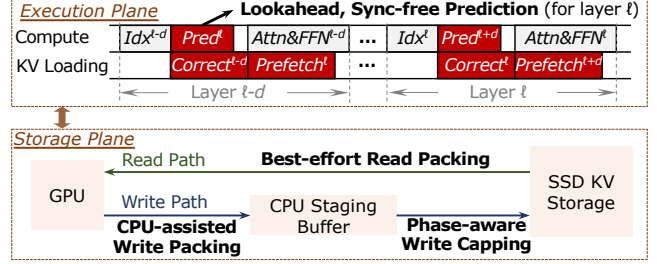


Figure 4: Overview of Janus.

selected KV caches. When execution reaches the target layer, its indexer computes the exact KV selection. Janus issues corrective reads for missing KV caches before executing sparse attention in order to preserve lossless model outputs. After append prefill, the request enters decode.

The storage plane provisions SSD I/O optimization techniques tailored for both read and write paths by taking the characteristics of sparse LLM agentic workloads into account (§4.3). For predictive and corrective reads, which involve non-contiguous GPU-SSD reads, Janus applies *best-effort read packing* to merge adjacent blocks whenever possible. To offload the newly generated KV caches, Janus leverages a *CPU-assisted write packing* approach, which allows for contiguous CPU-SSD writes without competing model execution on the GPU. Moreover, to maintain read performance for agentic workloads, Janus employs a simple-yet effective *phase-aware write capping*, which limits background write bandwidth when reads are in flight.

4.2 KV Selection Prediction

Training-free lookahead prediction. As mentioned in §3, prefetching from SSD requires a sufficient overlap window. Janus achieves this by invoking the target layer’s own indexer before its normal execution point. Denote the prediction distance as d , i.e., predicting KV selection for the sparse attention of layer ℓ at an earlier layer $\ell - d$. Let $\mathbf{h}_t^{(\ell-d)}$ denote the per-token hidden state at layer $\ell - d$. Janus feeds this earlier hidden state to the target layer’s indexer while retaining the target layer’s indexer keys $\mathbf{z}_s^{(\ell)}$. Formally, our lookahead prediction can be written as

$$\hat{\mathcal{S}}_t^{(\ell)} = \text{Idx}^{(\ell)} \left(\mathbf{h}_t^{(\ell-d)}, \left\{ \mathbf{z}_s^{(\ell)} \right\}_{s=1}^P; k \right) \quad (2)$$

For each target layer, Janus prefetches the union $\hat{\mathcal{S}}^{(\ell)} = \bigcup_{t \in [L]} \hat{\mathcal{S}}_t^{(\ell)}$ while the normal model execution proceeds. When layer ℓ later computes the exact KV selection $\mathcal{S}^{(\ell)}$, Janus issues corrective reads for selected entries absent from the predicted union, i.e., $\mathcal{S}^{(\ell)} \setminus \hat{\mathcal{S}}^{(\ell)}$. Thus, the amount of SSD I/O exposed on the critical path is reduced, while the KV caches

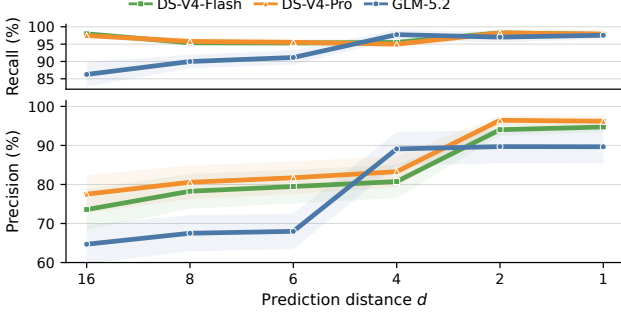


Figure 5: Prediction quality w.r.t. prediction distance (d). A higher recall indicates fewer corrective reads, and a higher precision indicates less redundancy in predictive reads.

consumed by sparse attention remain unaltered, ensuring loss-less model output. The lookahead prediction introduces no auxiliary model or parameters, requires no training or offline calibration, and turns the distance d into an overlap window for predictive reads.

Model-aligned prediction distance. We empirically evaluate how the prediction distance d affects the precision and recall of the predicted KV selection. Recall measures the fraction of the exact KV union covered by predictive reads (i.e., $|\hat{S}^{(\ell)} \cap S^{(\ell)}|/|S^{(\ell)}|$) and thus determines the amount of corrective reads, whereas precision measures the fraction of prefetched KV caches that are useful (i.e., $|\hat{S}^{(\ell)} \cap S^{(\ell)}|/|\hat{S}^{(\ell)}|$) and thus reflects the redundancy in predictive reads. Figure 5 reports the results across three models. Overall, a smaller d substantially improves precision, while recall is comparatively less sensitive and remains high, especially for DeepSeek-V4. Moreover, both metrics eventually saturate at short prediction distances, exhibiting diminishing returns.

It is noteworthy that the high recall at a large d does not necessarily indicate sound prediction. Although the indexer makes top- k selection for each of the L appended tokens, because semantically related tokens tend to have overlapping selection, the union of selection is typically smaller than $L \times k$. At a large d , the top- k prediction for each appended token becomes more random, which leads to less overlapping in the selection, inflating the predicted union. This larger union can cover many exact tokens through broader coverage and thereby preserve recall, but it also redundantly fetches many KV caches that are never used, as reflected by the low precision. Consequently, we seek the prediction distance that achieves high precision and recall.

Figure 5 further reveals that the point of diminishing returns aligns with model designs. For both DeepSeek-V4 models, reducing d from 2 to 1 changes the two metrics by less than 0.7%. This saturation distance aligns with their interleaved sparse–dense attention architecture (i.e., consecutive sparse attention layers are separated by two layers). For GLM-5.2, the saturation distance is 4, aligning with its IndexShare design

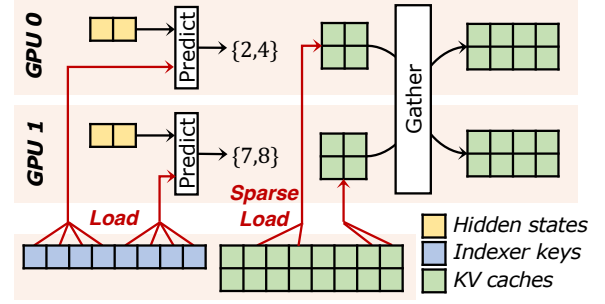


Figure 6: Illustration of our synchronization-free prediction.

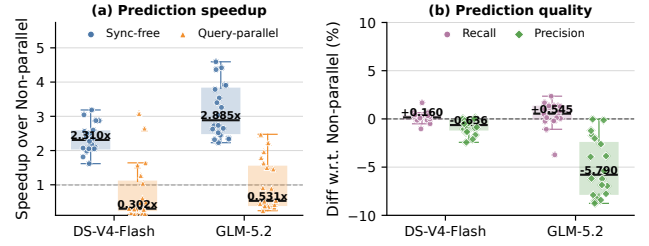


Figure 7: Assessment of synchronization-free prediction on 8 GPUs. (a) Prediction speedup over non-parallel prediction of synchronization-free and query-parallel prediction methods. (b) The differences in prediction quality of synchronization-free prediction compared to non-parallel prediction.

that shares top- k selections across four consecutive layers.

Motivated as such, we set $d = 2$ for DeepSeek-V4 and $d = 4$ for GLM-5.2, provisioning sound prediction quality while allowing predictive reads to be issued as early as possible. By doing so, the recall metric is above 96% for all three models, indicating that there are only 4% of the sparse KV loading remaining on the critical path. Meanwhile, we keep the redundancy in predictive reads below 5% for DeepSeek-V4 and 10% for GLM-5.2.

Efficient synchronization-free parallel prediction. Although our lookahead prediction creates the overlap window for prefetching, it incurs non-negligible computation overhead. To be specific, it requires to compute the relevance scores for L appended tokens against their preceding tokens, resulting in a computation complexity of $O(PL)$ given that $P > L$. Such prediction overhead would harm the overall performance. Considering that frontier LLMs are commonly deployed on multiple GPUs due to their model scale, we manage to distribute prediction across these GPUs so that each GPU performs approximately $1/N$ of the computation, where N is the number of GPUs to deploy one model replica.

However, how to efficiently parallelize the prediction necessitates thoughtful deliberation. Below we analyze two natural partitioning dimensions.

- *Query-parallel partitioning* distributes the hidden states

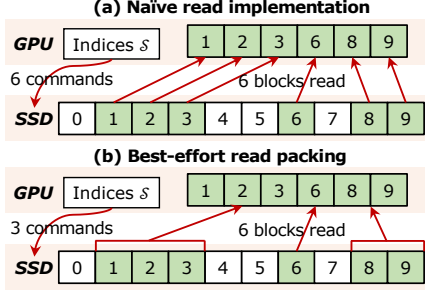


Figure 8: Illustration of read packing.

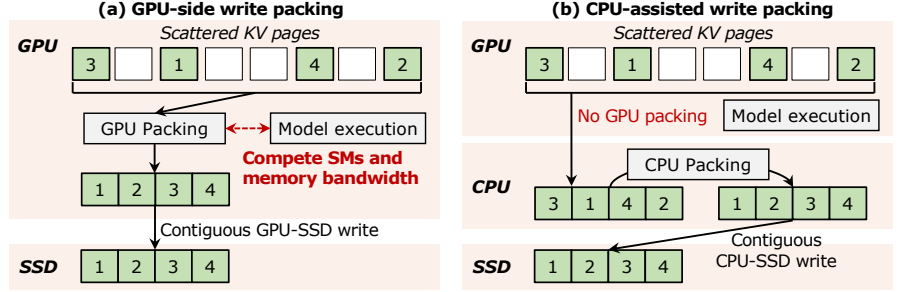


Figure 9: Illustration of GPU-side write packing and CPU-assisted write packing.

$\mathbf{h}^{(\ell-d)}$ for the L appended query tokens across GPUs. Each GPU predicts the selection for its assigned query tokens, after which the GPUs must merge and deduplicate their selected indices before prefetching to avoid redundant reads.

- *Indexer-key-parallel partitioning* shards the indexer keys $\mathbf{z}^{(\ell)}$ for the P cached history tokens. Each GPU computes the relevance scores for all appended tokens against one history shard and produces a local top- k list for each appended token. Then, the GPUs gather the local lists (along with the local top- k relevance scores) to obtain the exact global top- k list for each appended token, and finally obtain the union of selected tokens.

As a result, although both partitioning approaches can reduce the computation on each GPU, they incur a cross-GPU synchronization point between the prediction and SSD prefetching. Although the communication volume among the GPUs is not high, synchronization still prevents us from speeding up prediction via parallelization.

Fortunately, we find that our lookahead prediction can be synchronization-free under index-key-parallel partitioning. This is because speculative prefetching does not require the exact global top- k . Motivated by this, we develop a synchronization-free parallel prediction by turning the global top- k computation into N individual top- k/N processes across the GPUs, as shown in Figure 6.

Formally, denote C_n as the history shard assigned to GPU n . We first let each GPU produce the local top- k/N list for each appended token based on the corresponding history shard, i.e.,

$$\hat{\mathcal{S}}_t^{(\ell,n)} = \text{Idx}^{(\ell)} \left(\mathbf{h}_t^{(\ell-d)}, \left\{ \mathbf{z}_s^{(\ell)} \right\}_{s \in C_n}; k/N \right) \quad (3)$$

Then, instead of synchronizing the local results into the global one, GPU n directly computes the union of local selected indices $\hat{\mathcal{S}}^{(\ell,n)} = \bigcup_{t \in [L]} \hat{\mathcal{S}}_t^{(\ell,n)}$. Subsequently, each GPU immediately triggers the prefetching individually. Since different shards cover disjoint history tokens, there will be no duplicated reads. Finally, all GPUs gather the locally selected states using IPC before the target layer consumes them.

It is undoubted that removing the synchronization cannot ensure exact global top- k selection for each appended

token, since a certain shard may contain more than k/N high-scoring history tokens. However, after combining the token selection for all appended tokens, the approximation errors become minor. Our empirical results in Figure 7 show that, the synchronization-free parallel prediction achieves almost the same recall compared to non-parallel prediction, with a small drop in precision. In terms of efficiency, our approach consistently accelerates the prediction, achieving speedup by $2.8\times$ and $2.3\times$ on average for GLM-5.2 and DeepSeek-V4-Flash, respectively (results on both DeepSeek-V4 models are similar). In contrast, the query-parallel approach slows down the prediction in many cases due to synchronization.

Last but not least, our approach further helps reduce the SSD I/O per GPU. As depicted in Figure 6, each GPU only needs to load its own history shard and its local selected KV caches, rather than loading the full KV history. Therefore, Janus parallelizes both prediction and SSD reads.

4.3 SSD I/O Optimization

In this section, we introduce how Janus addresses the non-contiguous I/O and read-write interference for serving sparse attention LLMs under agentic workloads.

Best-effort read packing. As mentioned in §2.3, to initiate an SSD read on the GPU side, the GPU needs to enqueue I/O commands into the NVMe SQ and then rings the doorbell. A naïve implementation maps every selected SSD block to an individual command, as shown in Figure 8(a). Although the GPU can trigger these commands with a single doorbell ring, the SSD still executes many small reads, which underutilize its bandwidth and incur per-command processing overhead.

Janus instead converts the KV selection into an address-ordered list of unique SSD blocks and identifies maximal runs of consecutively selected blocks. As illustrated in Figure 8(b), Janus submits one read command for each run rather than one command for each block. For example, three selected blocks at consecutive addresses are fetched with one request covering all three blocks. This packing is best-effort: Janus merges only blocks that are both selected and physically adjacent, without fetching unselected blocks merely to enlarge a request. Consequently, read packing increases the average request size

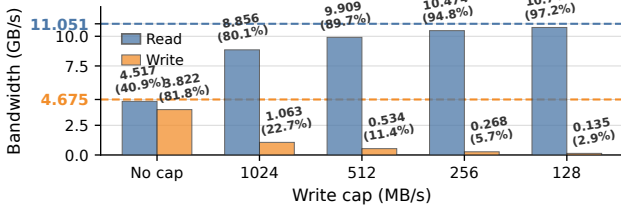


Figure 10: The read and write bandwidth under different write caps. The blue and orange lines indicate the read and write bandwidth without interference, respectively. Values in parentheses are the portion w.r.t. bandwidth without interference.

while reducing the number of NVMe commands as well as their pressure on the device queues. When the selected blocks are fully scattered, it naturally falls back to the original block-granular reads without introducing additional I/O.

CPU-assisted write packing. Paged attention allocates the KV caches of newly generated tokens to non-contiguous GPU pages [14]. Directly offloading these pages to SSD requires many small, scattered writes, which not only underutilize SSD bandwidth but also consume NVMe queue entries and repeatedly incur the fixed latency of an I/O request [28]. A natural alternative is to relay the pages into a contiguous GPU buffer before issuing a large GPU-to-SSD write, as shown in Figure 9(a). However, this relay requires a GPU kernel that competes with model execution for streaming multiprocessors (SMs) and memory bandwidth, which is undesirable.

Janus therefore offloads write packing to the CPU, as illustrated in Figure 9(b). When KV offloading occurs, Janus first transfers the scattered KV pages asynchronously to a bounded CPU staging buffer. The CPU then relays these pages in token order, after which Janus writes the resulting contiguous buffer to the SSD with one sequential request. This removes packing work from the GPU and avoids scattered page-granular SSD writes. Moreover, the resulting token-ordered SSD placement creates the contiguous regions that best-effort read packing can exploit in subsequent append prefills.

It is noteworthy that the CPU staging buffer does not rely on a large CPU DRAM space. Our empirical results in §5 show that only 16 GiB of CPU staging buffer is already sufficient for serving large-scale models like DeepSeek-V4-Pro, which has 1.6 trillion parameters. The buffer holds only KV caches currently moving from GPU memory to SSD and is recycled after their writes complete. Therefore, CPU-assisted packing avoids the interference between GPU-side packing and model computation, while retaining the cost-efficiency advantage of SSD-centric KV storage.

Phase-aware write capping. Unfortunately, there exists interference between SSD reads and writes. Concurrent requests compete for device-internal resources, particularly the NVMe internal cache, and reduce both read and write bandwidth [28].

As shown in Figure 10, when read and write interfere with each other, read bandwidth is degraded by nearly 60%, which is more severely than write bandwidth is. Thus, this interference is especially harmful to agentic serving due to the read amplification described in §3. Moreover, prefetch reads must be overlapped within the time window, and corrective reads issued after the exact indexer lie directly on the critical path. Write traffic that slows down reads would therefore degrade the append prefill efficiency substantially.

To mitigate this interference, Janus applies a simple yet effective phase-aware write limit. Specifically, we observe that by capping the write bandwidth, we can restore read bandwidth concurrent write traffic. Motivated by this, during append prefill, Janus caps background writes at 512 MB/s per SSD, preserving about 90% of the read bandwidth for prefetch and corrective reads. To be specific, we split the write message into chunks of 256MB. Whenever the CPU is about to write a chunk, it probes whether there are inflight SSD reads. If yes, the write cap is enforced, and vice versa. By doing so, Janus prioritizes the read efficiency without permanently sacrificing write throughput.

4.4 Implementation

Janus is an agentic serving framework for sparse attention LLMs with SSD-centric KV storage, implemented with 44K lines of Python and C++/CUDA codes. We integrate Tutti [28] to support GPU-initiated SSD I/O while CPU-SSD writes are implemented using standard POSIX file I/O. Janus uses vLLM [41] as the backend engine for LLM inference and serving, yet other engines like SGLang [50] are also compatible. Janus supports frontier sparse attention LLMs including DeepSeek-V4 series and GLM-5 series models [6, 9]. We perform layer-wise prefetching for deterministic loading, including indexer keys, KV caches for dense attention layers, and the sliding window KV caches used in DeepSeek-V4. For sparsely selected KV caches, we speculatively prefetch them using our lookahead prediction and trigger corrective reads immediately after the indexer computes the exact KV selection. Furthermore, we invoke lookahead selection prediction during corrective reads (as depicted in Figure 4) to hide the prediction overhead, and we leverage GPU copy engines to perform the KV gathering after each GPU loads its own shard, avoiding SM contention with model execution.

5 Experiments

5.1 Experimental Setup

Hardware environments. We conduct experiments on one GPU server equipped with 8 H200 GPUs (each with 141 GB HBM), 1TB CPU DRAM, and 8 KIOXIA CD8P-R 7.68 TB SSDs. The GPUs are interconnected with 900 GB/s NVLink, and the other interconnects use PCIe 5.0.

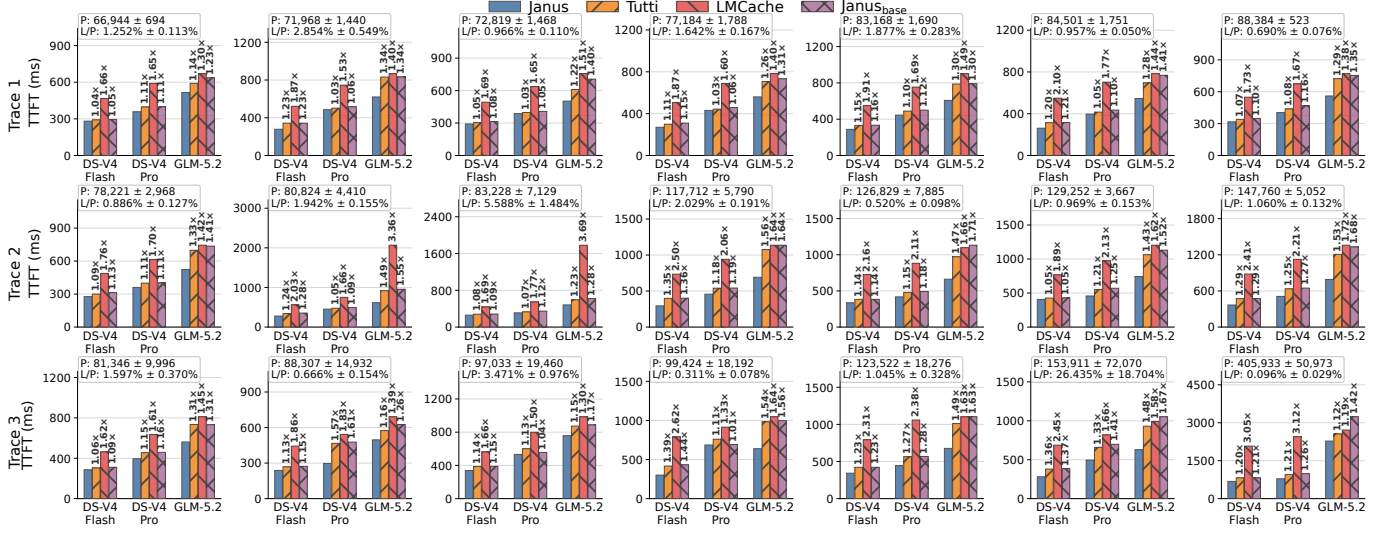


Figure 11: End-to-end TTFT for isolated append-prefill requests. For each trace, we cluster requests into seven groups by history length P and append-to-history ratio L/P , shown above each panel, and report the mean TTFT using bar plots within each group. The value above each bar is the speedup achieved by Janus compared to the corresponding baseline.

Models. Three frontier open-source LLMs are used in our experiments, which are DeepSeek-V4-Flash, DeepSeek-V4-Pro, and GLM-5.2 [6, 49], containing 284B, 1.6T, and 753B parameters, respectively. Model parameters and KV states are stored in FP8, while indexer keys of DeepSeek-V4 are stored in FP4 following their default option [6].

Traces. Our evaluation uses the three agentic traces listed in Table 2. The first two traces are generated³ from SWE-Bench-Pro and Terminal-Bench [7, 25], while the third one is a proprietary trace collected from our industrial partner.

Baselines. We mainly compare Janus against Tutti [28] and LMCache [22]. Tutti is an SSD-centric KV storage for LLM serving, which allows GPU to initiate SSD reads and writes without CPU intervention. LMCache is a widely used KV management framework that supports GPU-CPU-SSD hierarchy, but requires CPU to initiate each I/O request. In addition, since both Tutti and LMCache only support loading the full history KV at once, we further consider a variant of Janus, denoted as Janus_{base}. This variant does not adopt prediction, but trigger sparse KV reads only after the indexer finishes KV selection. All competitors use vLLM [41] as the backend, and we restrict available CPU DRAM capacity to be 16 GB to focus our evaluation on SSD storage. Specifically, we configure LMCache to use SSDs only, as hierarchical storage has much worse performance under the DRAM strict.

Evaluation metrics. Since we focus on the efficiency of

append prefill, we mainly report time to first token (TTFT) to demonstrate how Janus accelerates append prefill. We also report time per output token (TPOT) when evaluating Janus in serving to show that our work maintains decode efficiency. All experiments are averaged over five runs.

5.2 End-to-end Comparison

Append prefill efficiency. We first assess the time cost of append prefill. For each trace, we cluster append prefill requests with similar history lengths and append-to-history ratios because both dimensions affect the computation and sparse KV reads. We then compare the mean TTFT of all competitors.

As shown in Figure 11, the speedup achieved by Janus varies among clusters because history length and append ratio change both the sparse KV working set and the amount of model computation available for overlap. Nevertheless, Janus consistently achieves the lowest TTFT across all length characteristics. Specifically, Janus provides up to $1.57\times$ ($1.22\times$ on average) speedup over Tutti, up to $3.69\times$ ($1.85\times$ on average) speedup over LMCache, and up to $1.71\times$ ($1.27\times$ on average) speedup over Janus_{base}.

As LMCache relies on CPU to initiate GPU-SSD reads and writes, it suffers from the highest TTFT in most cases due to the extra framework-side overhead. Tutti and both variants of Janus support GPU-initiated SSD I/O to eliminate the related overhead. Tutti fetches all history KV while Janus_{base} loads only the sparsely selected KV. Theoretically speaking, Tutti results in redundant I/O compared with Janus_{base}. However, we observe that Janus_{base} has higher TTFT in many cases. This is not surprising: Janus_{base} leaves all sparse KV reads on

³Existing publicly available traces for agentic workloads (e.g., [1, 53]) contain per-round length information but do not provide raw texts/tokens, which are necessary for computing the sparse KV selection. Thus, we generate traces from benchmarks and we will open-source them.

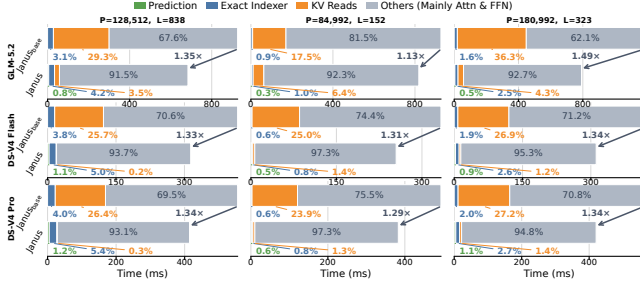


Figure 12: Time breakdown for three representative append prefill requests with different history lengths (P) and append lengths (L). Prefetched KV reads overlap with computation and are omitted from Janus’s bars; its KV-read segments contain only corrective reads. Green arrows show the speedup over Janus_{base}.

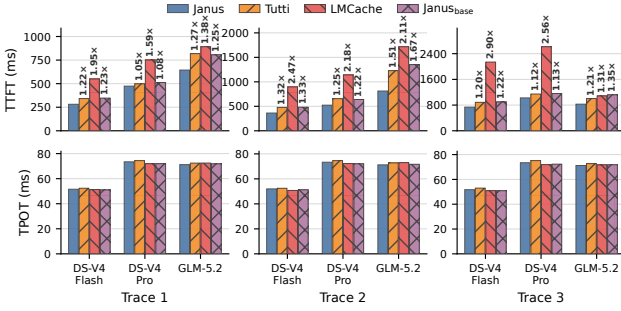


Figure 13: P95 TTFT and TPOT under end-to-end serving. The value above each bar is the speedup achieved by Janus compared to the corresponding baseline.

the critical path while the redundant loading of Tutti can be partially overlapped with model computation. For Janus, although corrective reads still lie on the critical path, by speculatively prefetching the sparsely selected KV, Janus successfully hides most SSD I/O with model computation, and therefore outperforms all baselines in terms of TTFT latency.

Case studies. To understand where the TTFT reduction comes from, Figure 12 decomposes the critical path of three representative append prefill requests across the three models. Janus_{base} first executes the exact indexer, then reads the selected sparse KV caches before attention can proceed. These reads account for 17.5–36.3% of the time cost of append prefill. Thus, even GPU-initiated reads expose a substantial non-compute interval when they start only after the exact selection becomes available.

Janus converts most of this interval into overlapped I/O. Its prediction runs before the target layer, allowing predicted KV reads to proceed concurrently with preceding computation. The exact indexer subsequently validates the prediction, and only missing KV caches require corrective reads. As a result, the corrective reads only consume 0.2–6.4% of the

Table 3: Ablation studies of our best-effort read packing and CPU-assisted write packing techniques (DeepSeek-V4-Flash, Trace 2). We show the relative degradation in P95 TTFT and TPOT compared with the setting in which both our read and write packing techniques are enabled.

Read	Write	TTFT	TPOT
No packing	CPU-assisted (ours)	4.2%↓	0.3%↓
Best-effort (ours)	No packing	3.1%↓	5.5%↓
Best-effort (ours)	GPU packing	2.7%↓	3.7%↓

time cost of append prefill. Model computation consequently account for 91.5–97.3% of Janus’s append prefill time cost, compared with 62.1–81.5% of Janus_{base}. The portion of corrective reads varies across models and requests, leading to divergent speedup. For both DeepSeek-V4 variants, corrective reads occupy at most 1.4% of the critical path, producing consistent speedups of 1.29–1.34×. GLM-5.2 exposes 3.5–6.4% time for corrective reads time and therefore shows a wider 1.13–1.49× speedup range.

Prediction itself adds little overhead. It contributes only 0.3–1.2% of the time cost of append prefill across the nine cases. As mentioned in §4.4, our implementation manages to overlap corrective reads and prediction with each other. However, when the prediction achieves high recall, the corrective reads may not be able to hide the prediction overhead. Thus, we place both of them on the critical path in Figure 12. Despite this, the prediction overhead is extremely small. Besides, we can also observe that the time cost of prediction is smaller than exact indexer, which verifies the effectiveness of our synchronization-free parallel prediction method.

Serving efficiency. Although Janus focuses on append prefill efficiency, we also conduct experiments under end-to-end serving. Specifically, we send the requests to serving frameworks following a Poisson process with a request arrival rate of 0.005, and report P95 TTFT and P95 TPOT in Figure 13.

In end-to-end serving, TTFT also involves the time cost of queuing, so the speedup range becomes narrower compared to the results in Figure 11. Nevertheless, Janus still outperforms Tutti by 1.05–1.51×, LMCACHE by 1.31–3.04×, and Janus_{base} by 1.01–1.68× in terms of P95 TTFT.

Meanwhile, the TPOT latencies of all competitors differ by approximately 3%, which is reasonable since we do not alter the decode computation. Tutti slightly increases TPOT in some cases, which is because it packs the KV pages on GPUs before offloading them to SSDs and thereby leads to SM contention. In contrast, Janus leverages CPU-assisted write packing to avoid competing SMs with model execution. In short, Janus accelerates append prefill substantially while maintaining decode efficiency.

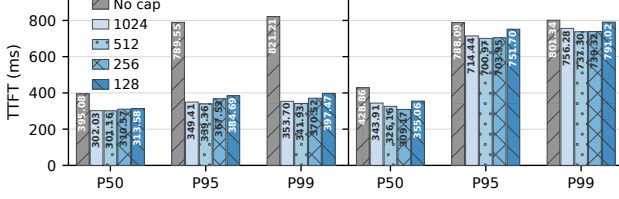


Figure 14: Percentile TTFT latencies when setting different write capping limits (DeepSeek-V4-Flash, Traces 2 and 3).

5.3 Effectiveness of SSD I/O Optimization

Read and write packing. We assess the effectiveness of our packing approaches that address the non-contiguous I/O issues. As shown in Table 3, when our best-effort read packing is disabled, the P95 TTFT latency is degraded by 4.2%, showing that the performance of sparse SSD reads is still important even though we can reduce the I/O exposed on critical path. For SSD writes, we compare with two approaches. The first directly triggers non-contiguous GPU-SSD writes without packing (denoted as “No packing”), whereas the second performs packing on GPUs and then triggers contiguous GPU-SSD writes (denoted as “GPU packing”). It can be seen that both approaches degrade P95 TPOT performance by 5.5% and 3.7%, respectively. The GPU packing approach enables contiguous SSD writes but competes GPU SMs with model execution, while the no packing approach suffers from non-contiguous I/O. In contrast, our CPU-assisted packing approach facilitates contiguous I/O while avoiding SM contention. Besides, the degradation in TPOT also harms TTFT due to the read-write interference in SSDs.

Write bandwidth cap. Figure 14 evaluates the write bandwidth cap using percentile TTFT. Without write capping, it suffers from severe read-write interference, bumping P99 TTFT substantially compared with a 512 MB/s cap. Meanwhile, an overly tight cap also reduces serving efficiency. With a write cap of 128 MB/s, P99 TTFT is 16.2% and 7.3% higher than that of 512 MiB/s. This is because the limited write bandwidth slows down KV offloading, causing pending KV writes to accumulate and increasing queueing time. In contrast, Janus consistently achieves sound TTFT performance with moderate caps. Thus we do not need to laboriously tune the write cap, demonstrating the robustness of our method.

6 Related Work

Hierarchical and SSD-backed KV storage. Existing systems extend KV capacity through hierarchical GPU-CPU-SSD or even remote storage, often coupled with cache-aware scheduling and overlapped transfers [8, 11, 22, 27, 37, 46]. As discussed in §1, owing to the explosive growth of LLM demands, CPU DRAM has become more and more costly

recently, which inspires us to focus on SSD-centric KV storage solutions. A line of research improves the GPU-SSD path through direct or KV-oriented storage abstractions [17, 28–30]. These systems assume that a reusable prefix identifies all required KV caches before prefill, whereas sparse attention reveals a query-dependent subset only during layer execution. Janus therefore coordinates sparse selection with SSD I/O instead of loading the full cached prefix.

Sparse-attention LLMs and serving. Research on sparse attention either retrofits dense models with token-, page-, or pattern-level selection [12, 31, 38], or trains sparse selection into the model itself [5, 6, 23, 48, 49]. Related mechanisms, including IndexCache and model-native IndexShare, reuse selection results across layers [3, 49]. Janus does not alter these algorithms. Instead, it serves models with native sparse selection while preserving their exact outputs.

Several systems manage sparse KV placement between GPU and CPU memory without predicting future demand [45, 52]. Predictive systems instead speculate future KV selections to hide transfer latency [15, 18, 21, 51]. IMPRESS, InfiniGen, SparseServe, and SolidAttention derive sparse access from originally dense-attention LLMs and would change their outputs [4, 15, 51, 52]. Most predictive designs also target decode, where adjacent-step similarity enables prediction. ECHO [20] supports prefetching for prefill of sparse attention, but it provides insufficient overlap for SSD latency, as discussed in §3. Janus provides a longer lookahead for append prefill, corrects prediction misses before attention, and co-designs fragmented SSD reads with background KV writes.

Agentic serving systems. Agentic serving systems exploit request dependencies, tool lifetimes, or session locality to schedule programs and retain reusable KV caches [10, 13, 16, 19, 24, 39, 42–44]. Workload studies show that agents repeatedly append to long contexts while producing relatively short outputs [2, 53]. Orthogonal to these scheduling and placement policies, Janus optimizes the inference path of each append prefill by discovering and fetching its query-dependent sparse KV caches from SSD.

7 Conclusion

In this work, we presented Janus, a novel agentic serving framework for sparse attention LLMs with SSD-centric KV storage. We first characterized agentic workloads, revealing that the bottleneck of SSD reads occurs in append prefill and that read efficiency should be emphasized. Then, we developed an efficient lookahead prediction approach to speculatively prefetch KV caches while performing corrective reads to ensure lossless model outputs. We further tailored a series of SSD I/O optimization for sparse attention LLMs under agentic workloads. Empirical results show that Janus achieves up to $1.57\text{--}3.69\times$ ($1.22\text{--}1.85\times$ on average) in terms of the time to first token latency compared to existing works.

References

- [1] Agentx benchmark datasets. <https://inference.semianalysis.com/agentx>, 2026.
- [2] Longju Bai, Zheming Huang, Xingyao Wang, Jiao Sun, Rada Mihalcea, Erik Brynjolfsson, Alex Pentland, and Jiaxin Pei. How do AI agents spend your money? analyzing and predicting token consumption in agentic coding tasks. *arXiv preprint arXiv:2604.22750*, 2026.
- [3] Yushi Bai, Qian Dong, Ting Jiang, Xin Lv, Zhengxiao Du, Aohan Zeng, Jie Tang, and Juanzi Li. Indexcache: Accelerating sparse attention via cross-layer index reuse. *arXiv preprint arXiv:2603.12201*, 2026.
- [4] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. IMPRESS: An importance-informed multi-tier prefix KV storage system for large language model inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 187–201, 2025.
- [5] DeepSeek-AI. DeepSeek-V3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [6] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. *arXiv preprint arXiv:2606.19348*, 2026.
- [7] Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, Karmini Sampath, Maya Krishnan, Srivatsa Kundurthy, Sean Hendryx, Zifan Wang, Chen Bo Calvin Zhang, Noah Jacobson, Bing Liu, and Brad Kenstler. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *CoRR*, abs/2509.16941, 2025.
- [8] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*, pages 111–126, 2024.
- [9] GLM-5 Team. GLM-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- [10] Wenhao He, Youhe Jiang, Penghao Zhao, Quanqing Xu, Eiko Yoneki, Bin Cui, and Fangcheng Fu. Efficient multi-round LLM inference over disaggregated serving. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
- [11] Cunchen Hu, Heyang Huang, Junhao Hu, Jiang Xu, Xusheng Chen, Tao Xie, Chenxi Wang, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. MemServe: Context caching for disaggregated LLM serving with elastic memory pool. *arXiv preprint arXiv:2406.17565*, 2024.
- [12] Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, et al. MInference 1.0: Accelerating pre-filling for long-context LLMs via dynamic sparse attention. *arXiv preprint arXiv:2407.02490*, 2024.
- [13] Hao Kang, Ziyang Li, Xinyu Yang, et al. ThunderAgent: A fast, simple, and program-aware agentic inference system. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026.
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PageAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023.
- [15] W. Lee et al. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024.
- [16] Hanchen Li, Runyuan He, Qiuyang Mang, et al. Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live. *arXiv preprint arXiv:2511.02230*, 2025.
- [17] Shaobo Li, Yirui Eric Zhou, Yuqi Xue, Yuan Xu, and Jian Huang. Managing scalable direct storage accesses for GPUs with GoFS. In *Proceedings of the 31st ACM Symposium on Operating Systems Principles (SOSP)*, pages 979–995, 2025.
- [18] Zuning Liang, Zhiyi Yao, Qi Chen, Yuedong Xu, Hao Dai, Zhiqiang Ding, Tongkai Yang, Jinlong Hou, and Yuan Cheng. DualDecoder: Accelerate long context LLM inference by predictive prefetch. *arXiv preprint arXiv:2607.26475*, 2026.
- [19] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of LLM-based applications with semantic variable. *arXiv preprint arXiv:2405.19888*, 2024.
- [20] Guangda Liu, Wenhao Chen, Chengwei Li, Zhenyu Ning, Jing Lin, Yiwu Yao, Quan Chen, Shixuan Sun, Jieru Zhao, and Minyi Guo. ECHO: Efficient KV cache offloading with lossless prefetching for serving native

- sparse attention LLMs. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, pages 17–37, 2026.
- [21] Guangda Liu, Chengwei Li, Zhenyu Ning, Jing Lin, Yiwu Yao, Danning Ke, Minyi Guo, and Jieru Zhao. Freekv: Boosting kv cache retrieval for efficient llm inference. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [22] Yuhan Liu, Yihua Cheng, Jiayi Yao, Yuwei An, et al. LMCACHE: An efficient KV cache layer for enterprise-scale LLM inference. *arXiv preprint arXiv:2510.09665*, 2025.
- [23] Enzhe Lu, Zhejun Jiang, Jingyuan Liu, et al. MoBA: Mixture of block attention for long-context LLMs. *arXiv preprint arXiv:2502.13189*, 2025.
- [24] Michael Luo, Xiaoxiang Shi, Colin Cai, et al. Autellix: An efficient serving engine for LLM agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- [25] Mike A. Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, Estefany Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis-Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *CoRR*, abs/2601.11868, 2026.
- [26] NVIDIA. GPUDirect Storage Overview Guide. NVIDIA Documentation, 2026. Accessed September 2026.
- [27] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a KVCache-centric architecture for serving LLM chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, 2025.
- [28] Shi Qiu, Yifan Hu, Xintao Wang, Wenhao Zhu, Jianqin Yan, Hao Chen, Kaiqiang Xu, Kai Chen, and Yiming Zhang. Tutti: Making SSD-backed KV cache practical for long-context LLM serving. *arXiv preprint arXiv:2605.03375*, 2026.
- [29] Shi Qiu, Weinan Liu, Yifan Hu, Jianqin Yan, Zhirong Shen, Xin Yao, Renhai Chen, Gong Zhang, and Yiming Zhang. GeminiFS: A companion file system for GPUs. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 221–236. USENIX Association, 2025.
- [30] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seung Won Min, Amna Masood, Jeongmin Park, Jinjun Xiong, CJ Newburn, Dmitri Vainbrand, I-Hsin Chung, Michael Garland, William Dally, and Wen mei Hwu. GPU-initiated on-demand high-throughput storage access in the BaM system architecture. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.
- [31] Luka Ribar, Ivan Chelombiev, Luke Hudliss-Galley, Charlie Blake, Carlo Luschi, and Douglas Orr. SparQ attention: Bandwidth-efficient LLM inference. *arXiv preprint arXiv:2312.04985*, 2023.
- [32] ServerSupply. Micron 6500 ION 30.72TB U.3 NVMe SSD. https://www.serversupply.com/SSD/NVMe/30.72TB/MICRON/MTFDKCC30T7TGR-1BK1DFCYY_377155.htm, 2026. Public advertised price: \$14,500; accessed September 2026.
- [33] ServerSupply. SK hynix HMCT14MHERC563N 256GB DDR5-6400 3DS RDIMM. https://www.serversupply.com/MEMORY/PC5-51200/256GB/HYNIX/HMCT14MHERC563N_429000.htm, 2026. Public advertised price: \$9,100; accessed September 2026.
- [34] ServerSupply. SK hynix HMCT14MHERC567N 256GB DDR5-6400 3DS RDIMM. https://www.serversupply.com/MEMORY/PC5-51200/256GB/HYNIX/HMCT14MHERC567N_429002.htm, 2026. Public advertised price: \$8,500; accessed September 2026.
- [35] ServerSupply. Solidigm D5-P5336 30.72TB PCIe 4.0 NVMe SSD. https://www.serversupply.com/SSD/NVMe/30.72TB/SOLIDIGM/SBFPF2BV307T001_376260.htm, 2026. Public advertised price: \$16,000; accessed September 2026.

- [36] ServerSupply. Supermicro MEM-DR525L-SL01-ER64 256GB DDR5-6400 ECC RDIMM. https://www.serversupply.com/MEMORY/PC5-51200/256GB/SUPERMICRO/MEM-DR525L-SL01-ER64_427546.htm, 2026. Public advertised price: \$14,350; accessed September 2026.
- [37] SGLang Team. Sglang hicache: Fast hierarchical kv caching with your favorite storage backends. <https://www.lmsys.org/blog/2025-09-10-sglang-hicache/>, 2025.
- [38] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. *arXiv preprint arXiv:2406.10774*, 2024.
- [39] Shubham Tiwari, Tapan Chugh, Nash Rickert, Simon Peter, Ratul Mahajan, and Haiying Shen. CacheWise: Understanding workloads and optimizing KVCache management for efficiently serving LLM coding agents. *arXiv preprint arXiv:2606.16824*, 2026.
- [40] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [41] vLLM Contributors. vllm: A high-throughput and memory-efficient inference and serving engine for llms. <https://github.com/vllm-project/vllm>, 2026.
- [42] Jiahao Wang, Kaizhan Lin, Kaixi Zhang, et al. SMetric: Rethink LLM scheduling for serving agents with balanced session-centric scheduling. *arXiv preprint arXiv:2607.08565*, 2026.
- [43] Yongtong Wu, Shaoyuan Chen, Yinmin Zhong, Rilin Huang, Yixuan Tan, Wentao Zhang, Liyue Zhang, Shangyan Zhou, Yuxuan Liu, Shunfeng Zhou, et al. DualPath: Breaking the storage bandwidth bottleneck in agentic LLM inference. *arXiv preprint arXiv:2602.21548*, 2026.
- [44] Tian Xia, Hanchen Li, Zhifei Li, Xiaokun Chen, Hao Kang, Yifan Qiao, Yi Xu, and Ion Stoica. Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI. *arXiv preprint arXiv:2606.00866*, 2026.
- [45] Zhiqiang Xie, Zhangheng Huang, Tingwei Huang, Ziyi Xu, Ruiyang Ma, and Christos Kozyrakis. HiSparse: Scaling sparse-attention decoding with hierarchical KV cache management. *arXiv preprint arXiv:2608.07009*, 2026.
- [46] Zhiqiang Xie, Ziyi Xu, Mark Zhao, Yuwei An, Vikram Sharma Mailthody, Scott Mahlke, Michael Garland, and Christos Kozyrakis. Strata: Hierarchical context caching for long context language model serving. In *20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26)*, 2026.
- [47] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*, pages 521–538, 2022.
- [48] Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. *arXiv preprint arXiv:2502.11089*, 2025.
- [49] Z.ai. GLM-5.2: Built for long-horizon tasks. <https://z.ai/blog/glm-5.2>, 2026.
- [50] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Livia Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems (NeurIPS 2024)*, 37:62557–62583, 2024.
- [51] Xinrui Zheng, Dongliang Wei, Jianxiang Gao, Yixin Song, Zeyu Mi, and Haibo Chen. SolidAttention: Low-latency SSD-based serving on memory-constrained PCs. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*, pages 67–82, 2026.
- [52] Qihui Zhou, Peiqi Yin, Pengfei Zuo, and James Cheng. SparseServe: Unlocking parallelism for dynamic sparse attention in long-context LLM serving. *arXiv preprint arXiv:2509.24626*, 2025.
- [53] Kan Zhu, Mathew Jacob, Chenxi Ma, Yi Pan, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. TraceLab: Characterizing coding agent workloads for LLM serving. *arXiv preprint arXiv:2606.30560*, 2026.